

1. Diseño de la Jerarquía de Clases

Para este sistema, definimos una estructura donde los vehículos comparten propiedades comunes, pero calculan su precio de forma diferenciada.

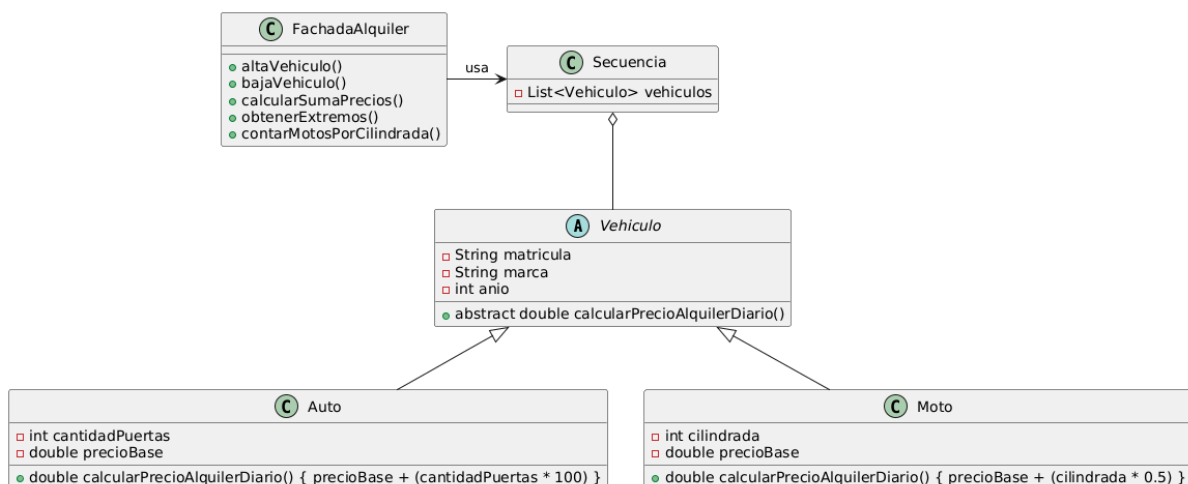
Definición de Tipos (Pseudocódigo)

- **Clase Abstracta Vehiculo**
 - Atributos: `matricula`, `marca`, `año`
 - Método (Abstracto): `CalcularPrecioAlquilerDiario()`
- **Clase Auto** (Hereda de `Vehiculo`)
 - Atributos adicionales: `cantidadPuertas`, `precioBase`
 - Método: `CalcularPrecioAlquilerDiario()`
- **Clase Moto** (Hereda de `Vehiculo`)
 - Atributos adicionales: `cilindrada`, `precioBase`
 - Método: `CalcularPrecioAlquilerDiario()`

Cada subclase implementa su propia lógica de negocio:

- **Lógica en Auto:** Retornar $\text{precioBase} + (\text{cantidadPuertas} * 100)$
- **Lógica en Moto:** Retornar $\text{precioBase} + (\text{cilindrada} * 0.5)$

Para modelar este sistema, aplicaremos el patrón **Fachada** (`FachadaAlquiler`) para encapsular la lógica de negocio y proteger la colección `Secuencia` de vehículos.



Código (plant UML):

```
startuml
```

```
abstract class Vehiculo {  
    -String matricula  
    -String marca  
    -int anio  
    +abstract double calcularPrecioAlquilerDiario()  
}
```

```
class Auto {  
    -int cantidadPuertas  
    -double precioBase  
    +double calcularPrecioAlquilerDiario() { precioBase + (cantidadPuertas * 100) }  
}
```

```
class Moto {  
    -int cilindrada  
    -double precioBase  
    +double calcularPrecioAlquilerDiario() { precioBase + (cilindrada * 0.5) }  
}
```

```
class Secuencia {  
    -List<Vehiculo> vehiculos  
}
```

```
class FachadaAlquiler {  
    +altaVehiculo()  
    +bajaVehiculo()  
    +calcularSumaPrecios()  
    +obtenerExtremos()  
    +contarMotosPorCilindrada()  
}
```

```

Vehiculo <|-- Auto

Vehiculo <|-- Moto

FachadaAlquiler -> Secuencia : usa

Secuencia o-- Vehiculo

@enduml

```

2. Desglose de Requerimientos (Pseudocódigo)

Requerimiento	Entrada	Salida	Proceso
1a. Alta	Datos (Matricula, marca, etc.)	Éxito/Error	Validar unicidad en Secuencia , instanciar y agregar.
1b. Baja	Matricula	Éxito/Error	Buscar por matricula en Secuencia y remover.
2. Suma	N/A	Total (double)	Iterar Secuencia , llamar calcularPrecioAlquilerDiario() (polimórfico).
3. Extremos	N/A	Vehículos	Recorrer y comparar retornos de calcularPrecioAlquilerDiario() .
4. Contar Moto	Cilindrada	Entero	Recorrer Secuencia , filtrar por tipo Moto y comparar atributo.

Requerimiento 1: Altas y Bajas

Procedimiento AltaVehiculo(datos):

Si MatriculaNoExiste(datos.matricula) Entonces:

Crear instancia de Auto o Moto con datos

Agregar a lista "Flota"

Sino:

Mostrar Error "Ya existe"

Procedimiento BajaVehiculo(matricula):

Eliminar de "Flota" el vehículo donde vehiculo.matricula == matricula

Requerimiento 2: Suma de precios

Funcion SumarPreciosDiarios():

total = 0

Para cada vehiculo en Flota:

total = total + vehiculo.CalcularPrecioAlquilerDiario()

Retornar total

Requerimiento 3: Mayor y Menor precio

Funcion ObtenerExtremosPrecios():

mayor = -1, menor = 999999

Para cada vehiculo en Flota:

precio = vehiculo.CalcularPrecioAlquilerDiario()

Si precio > mayor: mayor = precio

Si precio < menor: menor = precio

Retornar mayor, menor

Requerimiento 4: Contar motos por cilindrada

Funcion ContarMotos(cilindradaBuscada):

contador = 0

Para cada vehiculo en Flota:

Si vehiculo es del tipo "Moto" Y vehiculo.cilindrada == cilindradaBuscada:

contador = contador + 1

Retornar contador

3. Especificación de los Métodos Polimórficos

A. Clase Vehiculo (Abstracta)

- **Atributos:** matricula, marca, anio
- **Constructor:** Vehiculo(mat, marc, anio)
- **Selectores:** getMatricula(), getMarca(), getAnio()
- **Modificadores:** setMarca(), setAnio()
- **Método Polimórfico:** abstract double calcularPrecioAlquilerDiario()

B. Clase Auto

- **Atributos:** cantidadPuertas, precioBase
- **Constructor:** Auto(mat, marc, anio, puertas, precioBase)
- **Selectores/Modificadores:** getCantidadPuertas(), setCantidadPuertas(), getPrecioBase(), setPrecioBase()
- **Método Polimórfico:** calcularPrecioAlquilerDiario() { return precioBase + (cantidadPuertas * 100); }

C. Clase Moto

- **Atributos:** cilindrada, precioBase
- **Constructor:** Moto(mat, marc, anio, cil, precioBase)
- **Selectores/Modificadores:** getCilindrada(), setCilindrada(), getPrecioBase(), setPrecioBase()
- **Método Polimórfico:** calcularPrecioAlquilerDiario() { return precioBase + (cilindrada * 0.5); }

D. Clase FachadaAlquiler (Capa de Lógica)

- **Atributos:** Secuencia vehiculos
- **Métodos específicos:**
 - altaVehiculo(v): Delega a Secuencia.insertar(v) tras validar.
 - bajaVehiculo(mat): Delega a Secuencia.eliminar(mat).
 - calcularSumaPrecios(): Recorre la Secuencia y suma resultados polimórficos.
 - obtenerVehiculosExtremos(): Retorna los objetos con mayor/menor retorno de método polimórfico.
 - contarMotosPorCilindrada(cil): Filtra por tipo y compara atributo en Secuencia.

E. Clase Secuencia (Capa de Colección)

- **Atributos:** List<Vehiculo>
- **Métodos:** insertar(Vehiculo), eliminar(String matricula), obtenerTodos(), existeMatricula(String)